BHTTP1 Protocol Specification

Jaivardhan D. Rao | 24BCS10117 | Network Architecture

BHTTP/1 transfers static files over an IPv4 TCP connection. This document is the complete wire contract for separately implemented server and client programs. The endpoint is configured for BHTTP/1 in advance; there is no HTTP upgrade, TLS, handshake, dynamic compression, or multiplexing. Numeric values are unsigned and big-endian. All lengths count encoded bytes, not characters. MUST and MUST NOT are requirements.

Fixed frame header

Every frame consists of exactly 8 header bytes followed by its payload. TCP segment boundaries have no meaning; receivers MUST read the stated byte counts with a loop.

Byte offsets	Bits	Field	Definition
0 to 3	32	Payload length	Excludes the 8-byte header. Maximum 1,048,576 bytes.
4	8	Type	1 = REQUEST; 2 = RESPONSE; other values are extensions.
5	8	Flags	Send 0. Receivers ignore currently unassigned bits.
6 to 7	16	Request ID	REQUEST uses 1 to 65,535. RESPONSE echoes it.

The 32-bit length is easy to encode with ordinary integer packing and leaves capacity for a future profile; the v1 cap bounds allocation. An 8-bit type leaves extension space. The flags byte reserves optional future signals without changing the header. A 16-bit correlation ID is sufficient for sequential exchanges; it is not a multiplexed stream identifier. HTTP/2 uses a 9-byte header with 24/8/8/31-bit length/type/flags/stream fields plus a reserved bit. Its 24-bit length permits up to 16,777,215 payload bytes (default limit 16,384); 8-bit types and flags allow new frame kinds and per-kind options; 31-bit stream IDs support independent concurrent streams [1].

Ten predefined header names

ID	Name	Sent in the supplied example
1	host	Request
2	user-agent	Request
3	accept	Request
4	content-type	Response
5	content-length	Response
6	server	Response
7	connection	Request and response
8	cache-control	Response
9	x-request-id	Request and response
10	x-course	Request and response

These IDs are fixed for v1. New names use literal encoding, not new numeric IDs. The supplied response also sends the literal name x-file-name. Name indexing saves repeated name bytes; values remain literals. This is inspired by HPACK's indexed-name and literal-name representations, and is not full HPACK [2].

Header block encoding

Each block starts with count:u8, followed by count fields. Count is 0 to 32. A predefined name is encoded as ID:u8, valueLength:u16, valueBytes. A literal name is 0:u8, nameLength:u8, nameBytes, valueLength:u16, valueBytes. Names MUST be 1 to 64 lowercase ASCII HTTP-token characters (!#$%&'*+.^_`|~0-9a-z-). Values MUST be valid UTF-8, at most 4,096 bytes, with no ASCII controls except TAB; DEL is forbidden. Duplicate names and IDs 11 to 255 are malformed. A receiver accepts either name representation. There is no dynamic table or Huffman coding.

Request and response payloads

REQUEST payload: method:u8, pathLength:u16, pathBytes, headerBlock. Method 1 means GET; other methods produce 405. The UTF-8 path MUST begin with /, contain 1 to 4,096 bytes, and contain no raw ASCII control characters (0 to 31 or 127). Requests have no body. RESPONSE payload: status:u16, headerBlock, bodyLength:u32, bodyBytes. Status is 100 to 599. The content-length header MUST be the canonical decimal representation of bodyLength. No field may overrun the frame, and trailing bytes are malformed. Other headers are metadata and do not change framing.

Connection and extension rules

A client MUST open exactly one TCP connection per invocation and MUST NOT reconnect, retry, or follow redirects. It sends one request, consumes its response, then may send another request on the same connection. The supplied client uses IDs 1, 2, and so on, and limits one invocation to 65,535 URLs at the same origin. The server processes requests sequentially and keeps the connection open after a complete response. A receiver encountering an unknown frame type MUST consume exactly its advertised payload length, discard the frame, and continue at the next header without producing a response or changing request state. This applies at both endpoints, regardless of the unknown frame's flags or ID. The global frame-size limit still applies. Future compatible versions add frame types; incompatible header layouts need separate endpoint configuration.

Files and errors

For GET, reject authority or fragment syntax with 400, discard the query, percent-decode valid escapes once as UTF-8, and walk relative to the document root. Invalid percent escapes remain literal; invalid decoded UTF-8 gets 400. / maps to index.html. Dot and dot-dot segments, backslashes, decoded controls, and symlink targets MUST NOT be served. Only regular files are served, with a 1,000,000-byte file limit. Return 200 with the exact file bytes; 404 for absent files or directories; 403 for blocked paths or access; 405 for unsupported methods; 413 for the size limit; 500 for other file I/O failures.

A fully received malformed known frame gets a 400 RESPONSE with the available request ID, and the connection stays usable. An oversized length or EOF within a header/payload gets 413 or 400 respectively, followed by close; use ID 0 if no complete header supplied an ID. Response connection metadata is keep-alive normally and close for fatal framing errors. Clean EOF between frames needs no response. The supplied server closes idle connections after 30 seconds; the client times out after 5 seconds without retrying. A client rejects wrong response IDs, known frames in the wrong direction, and malformed responses. Exit codes are 0 for success, 1 if any status is 4xx/5xx, and 2 for usage, transport, or protocol errors. Response body bytes go only to stdout; -v dumps every transmitted and received frame to stderr.

References

1. RFC 9113, section 4.1: https://www.rfc-editor.org/rfc/rfc9113.html#section-4.1

2. RFC 7541, sections 2.3 and 6.2: https://www.rfc-editor.org/rfc/rfc7541.html